

NetworkTables

中英对照译文

按 WPILib 专题索引顺序收录十篇文章

Contents / 目录

01	What is NetworkTables
02	NetworkTables Tables and Topics
03	Publishing and Subscribing to a Topic
04	NetworkTables Instances
05	NetworkTables Networking
06	Listening for Changes
07	Writing a Simple NetworkTables Robot Program
08	Creating a Client-side Program
09	Migrating from NetworkTables 3.0 to NetworkTables 4.0
10	Reading Array Values Published by NetworkTables

What is NetworkTables / 什么是 NetworkTables

NetworkTables is an implementation of a [publish-subscribe messaging system](#). Values are published to named "topics" either on the robot, driver station, or potentially an attached coprocessor, and the values are automatically distributed to all subscribers to the topic. For example, a driver station laptop might receive camera images over the network, perform some vision processing algorithm, and come up with some values to sent back to the robot. The values might be an X, Y, and Distance. By writing these results to NetworkTables topics called "X", "Y", and "Distance" they can be read by the robot shortly after being written. Then the robot can act upon them. Similarly, the robot program can write sensor values to topics and those can be read and plotted in real time on a dashboard application.

NetworkTables 是一种[发布 / 订阅消息系统](#)的实现。机器人、Driver Station 或可能连接的协处理器可以向有名称的“主题”发布值，这些值会自动分发给该主题的所有订阅者。例如，Driver Station 笔记本电脑可以通过网络接收摄像头图像，执行某种视觉处理算法，并生成要发回机器人的若干值，例如 X、Y 和 Distance。将这些结果写入名为“X”“Y”“Distance”的 NetworkTables 主题后，机器人很快就能读到它们，并据此采取行动。同样，机器人程序也可以向主题写入传感器值，仪表盘应用程序能够实时读取并绘制这些值。

NetworkTables can be used by programs on the robot in Java, and is built into each version of WPILib.

机器人上的 Java 程序可以使用 NetworkTables；它内置于每个版本的 WPILib 中。

NOTE: NetworkTables has changed substantially in 2023. For more information on migrating pre-2023 code to use the new features, see [migrating from networktables 3.0 to networktables 4.0](#).

说明： NetworkTables 在 2023 年发生了重大变化。关于将 2023 年以前的代码迁移到新功能的详情，请参阅[从 NetworkTables 3.0 迁移到 NetworkTables 4.0](#)。

NetworkTables Concepts / NetworkTables 概念

First, let's define some terms:

首先定义几个术语：

- **Topic:** a named data channel. Topics have a fixed data type (for the lifetime of the topic) and mutable properties.
- **主题 (Topic)：** 具有名称的数据通道。主题在其整个生命周期内具有固定的数据类型，且属性可变。
- **Publisher:** defines the topic and creates and sends timestamped data values.
- **发布者 (Publisher)：** 定义主题，并创建、发送带时间戳的数据值。
- **Subscriber:** receives timestamped data value updates to one or more topics.
- **订阅者 (Subscriber)：** 接收一个或多个主题带时间戳的数据值更新。
- **Entry:** a combined publisher and subscriber. The subscriber is always active, but the publisher is not created until a publish operation is performed (e.g. a value is "set", aka published, on the entry). This may be more convenient than maintaining a separate publisher and subscriber.
- **条目 (Entry)：** 发布者和订阅者的组合。订阅者始终处于活动状态，但只有在执行发布操作时（例如在条目上“设置”一个值，即发布该值），才会创建发布者。相比分别维护发布者和订阅者，这可能更方便。
- **Property:** named information (metadata) about a topic stored and updated separately from the topic's data. A topic may have any number of properties. A property's value can be any data type that can be represented in JSON.
- **属性 (Property)：** 关于主题的具名信息（元数据），与主题数据分开存储和更新。一个主题可以有任意数量的属性。属性值可以是 JSON 能表示的任何数据类型。

NetworkTables supports a range of data types, including boolean, numeric, string, and arrays of those types. Supported numeric data types are single or double precision floating point, or 64-bit integer. There is also the option of storing raw data (an array of bytes), which can be used for representing binary encoded structured data. Types are represented as strings for efficiency reasons. There is also an enumeration for the most common types in the NetworkTables API.

NetworkTables 支持多种数据类型，包括布尔值、数值、字符串及其数组。支持的数值类型包括单精度或双精度浮点数，以及 64 位整数。还可以存储原始数据（字节数组），用于表示经过二进制编码的结构化数据。出于效率考虑，类型用字符串表示。NetworkTables API 还为最常用的类型提供了一个枚举。

Topics are created when the first publisher announces the topic and are removed when the last publisher stops publishing. It's possible to subscribe to a topic that has not yet been created/published.

第一个发布者宣布主题时，主题被创建；最后一个发布者停止发布时，主题被移除。可以订阅尚未创建或发布的主题。

Topics have properties. Properties are initially set by the first publisher, but may be changed at any time. Similarly to values, property changes to a topic are propagated to all subscribers to that topic. Properties are structured data (JSON), but at the top level are simply a key/value store (a JSON map). Some properties have defined behavior, but arbitrary ones can be set by the application.

主题具有属性。属性最初由第一个发布者设置，但之后可以随时更改。与值类似，主题属性的变化也会传播给该主题的所有订阅者。属性是结构化数据（JSON），但在顶层只是键值存储（JSON 映射）。部分属性具有预定义行为，应用程序也可以设置任意其他属性。

Publishers specify the topic's data type; while there can be multiple publishers to a single topic, they must all be publishing the same data type. This is enforced by the NetworkTables server (the first publisher "wins"). Typically single-topic subscribers also specify what data type they're expecting to receive on a topic and thus won't receive value updates of other types.

发布者指定主题的数据类型；虽然同一主题可以有多个发布者，但它们必须发布相同的数据类型。NetworkTables 服务器会强制执行这一要求（第一个发布者“获胜”）。通常，订阅单一主题的订阅者也会指定期望接收的数据类型，因此不会收到其他类型的值更新。

The [Network Tables Protocol Specification](#) contains detailed documentation on the current wire protocol.

[Network Tables Protocol Specification](#) 提供了当前网络传输协议的详细文档。

Retained and Persistent Topics / 保留主题与持久主题

While by default topics are transitory and disappear after the last publisher stops publishing, topics can be marked as retained (via setting the "retained" property to true) to prevent them from disappearing. For retained topics, the server acts as an implicit publisher of the last value, and will keep doing so as long as the server is running. This is primarily useful for configuration values; e.g. an autonomous mode selection published by a dashboard should set the topic as retained so its value is preserved in case the dashboard disconnects.

默认情况下，主题是临时的（transitory），在最后一个发布者停止发布后会消失。可以把主题标记为保留（retained）（将 `retained` 属性设为 `true`），以防它消失。对于保留主题，服务器充当最后一个值的隐式发布者，并在自身持续运行期间一直保持这种状态。这主要适用于配置值；例如，仪表板发布的自主模式选择应将主题设为保留，这样仪表板断开连接时该值仍会保存。

Additionally, topics can be marked as persistent via setting the "persistent" property to true. These operate similarly to retained topics, but in addition, persistent topic values are automatically saved to a file on the server and when the server starts up again, the topic is created and its last value is published by the server.

此外，可以将主题标记为持久（persistent）（将 `persistent` 属性设为 `true`）。它与保留主题的工作方式类似，但持久主题的值还会自动保存到服务器上的文件中；服务器重新启动后，会创建该主题并由服务器发布其上一次的值。

Value Propagation / 值的传播

The server keeps a copy of the last published value for every topic. When a subscriber initially subscribes to a topic, the server sends the last published value. After that initial value, new value updates are communicated to subscribers each time the publisher sends a new value.

服务器为每个主题保存最近一次发布的值。订阅者首次订阅某个主题时，服务器会发送最近一次发布的值。在这个初始值之后，每当发布者发送新值，新的值更新就会传送给订阅者。

NetworkTables is a client/server system; clients do not talk directly to each other, but rather communicate via the server. Typically, the robot program is the server, and other pieces of software on other computers (e.g. the driver station or a coprocessor) are clients that connect to it. Thus, when a coprocessor (client) publishes a value, the value is sent first from the coprocessor (client) to the robot program (server), and then the robot program distributes that value to any subscribers (e.g. the robot program local program, or other clients such as dashboards).

NetworkTables 是客户端 / 服务器系统；客户端之间不直接通信，而是通过服务器通信。通常，机器人程序是服务器，其他计算机上的软件（例如 Driver Station 或协处理器）是连接它的客户端。因此，当协处理器（客户端）发布一个值时，该值首先从协处理器（客户端）发送给机器人程序（服务器），随后由机器人程序分发给任何订阅者（例如机器人程序中的本地订阅者，或仪表板等其他客户端）。

The server does not send topic changes or value updates to clients that have not subscribed to the topic.

服务器不会向尚未订阅该主题的客户端发送主题变化或值更新。

By default, NetworkTables sends value updates periodically, batching the data to help limit the number of small packets being sent over the network. Also, by default, only the most recent value is transmitted; any intermediate value changes made between network transmissions are discarded. This behavior can be changed via publish/subscribe options--publishers and subscribers can indicate that all value updates should be preserved and communicated via the "send all" option. In addition, it is possible to force NetworkTables to "flush" all current updates to the network; this is useful for minimizing latency.

默认情况下，NetworkTables 会定期发送值更新，并将数据批量处理，以减少经网络发送的小数据包数量。此外，默认只传输最新的值；两次网络传输之间的中间值变化会被丢弃。可以通过发布 / 订阅选项更改此行为：发布者和订阅者可以使用“send all”选项，要求保留并传送所有值更新。还可以强制 NetworkTables 将当前所有更新“flush”到网络；这有助于降低延迟。

Timestamps / 时间戳

All NetworkTable value updates are timestamped at the time they are published. Timestamps in NetworkTables are measured in integer microseconds.

所有 NetworkTable 值更新都在发布时附上时间戳。NetworkTables 时间戳以整数微秒计量。

NetworkTables automatically synchronizes time between the server and clients. Each client maintains an offset between the client local time and the server time, so when a client publishes a value, it stores a timestamp in local time and calculates the equivalent server timestamp. The server timestamp is what is communicated over the network to any subscribers. This makes it possible e.g. for a robot program to get a reasonable estimation of the time when a value was published on a coprocessor relative to the current time.

NetworkTables 会自动同步服务器与客户端之间的时间。每个客户端都会维护其本地时间与服务器时间之间的偏移，因此客户端发布值时，会记录本地时间戳并计算对应的服务器时间戳。通过网络传给订阅者的是服务器时间戳。这样，机器人程序就能相对于当前时间，合理估计某个值是在协处理器上的什么时间发布的。

Because of this, two timestamps are visible through the API: a server timestamp indicating the time (estimated) on the server, and a local timestamp indicating the time on the client. When the RoboRIO is the NetworkTables server, the

server timestamp is the same as the FPGA timestamp returned by `Timer.getFPGATimestamp()` (except the units are different: NetworkTables uses microseconds, while `getFPGATimestamp()` returns seconds).

因此，API 中可以看到两个时间戳：一个是表示服务器上（估计）时间的服务器时间戳，另一个是表示客户端时间的本地时间戳。当 roboRIO 是 NetworkTables 服务器时，服务器时间戳与 `Timer.getFPGATimestamp()` 返回的 FPGA 时间戳相同（但单位不同：NetworkTables 使用微秒，而 `getFPGATimestamp()` 返回秒）。

NetworkTables Organization / NetworkTables 的组织结构

Data is organized in NetworkTables in a hierarchy much like a filesystem's folders and files. There can be multiple subtables (folders) and topics (files) that may be nested in whatever way fits the data organization desired. At the top level (`NetworkTableInstance`: [Java](#)), topic names are handled similar to absolute paths in a filesystem: subtables are represented as a long topic name with slashes ("/") separating the nested subtable and value names. A `NetworkTable` ([Java](#)) object represents a single subtable (folder), so topic names are relative to the NetworkTable's base path: e.g. for a root table called "SmartDashboard" with a topic named "xValue", the same topic can be accessed via `NetworkTableInstance` as a topic named `/SmartDashboard/xValue`. However, unlike a filesystem, subtables don't really exist in the same way folders do, as there is no way to represent an empty subtable on the network--a subtable "appears" only as long as there are topics published within it.

NetworkTables 按层级组织数据，与文件系统的文件夹和文件很相似。可以根据所需的数据组织方式，嵌套多个子表（文件夹）和主题（文件）。在顶层（`NetworkTableInstance`: [Java](#)），主题名称的处理方式类似文件系统中的绝对路径：子表表现为一个较长的主题名称，使用斜杠（/）分隔嵌套的子表和值名称。`NetworkTable` ([Java](#)) 对象表示单个子表（文件夹），因此主题名称相对于 NetworkTable 的基础路径。例如，对于名为“SmartDashboard”的根表及其中名为“xValue”的主题，也可以通过 `NetworkTableInstance` 使用主题名称 `/SmartDashboard/xValue` 访问同一主题。不过，与文件系统不同，子表并不像文件夹那样真正独立存在，因为网络上无法表示空子表；只有子表中有已发布主题时，它才会“出现”。

OutlineViewer is a utility for exploring the values stored in NetworkTables, and can show either a flat view (topics with absolute paths) or a nested view (subtables and topics).

OutlineViewer 是用于浏览 NetworkTables 中所存储值的工具，可以显示平铺视图（使用绝对路径的主题）或嵌套视图（子表和主题）。

There are some default tables that are created automatically when a robot program starts up:

机器人程序启动时，会自动创建一些默认表：

Table name / 表名	Use / 用途
<code>/SmartDashboard</code>	Used to store values written to the SmartDashboard or Shuffleboard using the <code>SmartDashboard.put()</code> set of methods. / 用于存储通过 <code>SmartDashboard.put()</code> 系列方法写入 SmartDashboard 或 Shuffleboard 的值。
<code>/LiveWindow</code>	Used to store Test mode (Test on the Driver Station) values. Typically these are Subsystems and the associated sensors and actuators. / 用于存储测试模式（Driver Station 中的 Test 模式）的值，通常包括 Subsystem 及其关联的传感器和执行器。
<code>/FMSInfo</code>	Information about the currently running match that comes from the Driver Station and the Field Management System / 来自 Driver Station 和场地管理系统、关于当前比赛的信息。

NetworkTables API Variants / NetworkTables API 变体

The object-oriented NetworkTables API in Java is recommended for robot code and general team use, and provides classes that help ensure correct use of the API.

Java 中面向对象的 NetworkTables API 推荐用于机器人代码和队伍的一般用途；它提供的类有助于确保正确使用 API。

Lifetime Management / 生命周期管理

Publishers, subscribers, and entries only exist as long as the objects exist.

发布者、订阅者和条目只会在其对象存在期间存在。

In Java, a common bug is to create a subscriber or publisher and not properly release it by calling `close()`, as this will result in the object lingering around for an unknown period of time and not releasing resources properly. This is less common of an issue in robot programs, as long as the publisher or subscriber object is stored in an instance variable that persists for the life of the program.

在 Java 中，一个常见错误是创建订阅者或发布者后，没有通过调用 `close()` 正确释放；这会使对象在未知的一段时间内继续存在，并且无法正确释放资源。在机器人程序中，只要把发布者或订阅者对象保存在整个程序生命周期内都存在的实例变量中，这个问题就较少发生。

NetworkTables Tables and Topics / NetworkTables 的表与主题

Using the NetworkTable Class / 使用 NetworkTable 类

The `NetworkTable` ([Java](#)) class is an API abstraction that represents a single "folder" (or "table") of topics as described in [networktables organization](#). The `NetworkTable` class stores the base path to the table and provides functions to get topics within the table, automatically prepending the table path.

`NetworkTable` ([Java](#)) 类是一种 API 抽象，表示[NetworkTables 的组织结构](#)中所述的单个主题“文件夹”（或“表”）。`NetworkTable` 类保存该表的基础路径，并提供获取表内主题的方法；这些方法会自动在主题名称前加上表路径。

Getting a Topic / 获取主题

A `Topic` ([Java](#)) object represents a topic. This has a 1:1 correspondence with the topic's name, and will not change as long as the instance exists. Unlike publishers and subscribers, it is not necessary to store a `Topic` object.

`Topic` ([Java](#)) 对象表示一个主题（topic）。它与主题名称一一对应，只要实例存在，这种对应关系就不会改变。与发布者 and 订阅者不同，不需要保存 `Topic` 对象。

Having a `Topic` object does not mean the topic exists or is of the correct type. For convenience when creating publishers and subscribers, there are type-specific `Topic` classes (e.g. `BooleanTopic`: [Java](#)), but there is no check at the `Topic` level to ensure that the topic's type actually matches. The preferred method to get a type-specific topic is to call the appropriate type-specific getter, but it's also possible to directly convert a generic `Topic` into a type-specific `Topic` class.

拥有 `Topic` 对象并不意味着该主题已经存在，或其类型正确。为便于创建发布者和订阅者，API 提供了特定类型的 `Topic` 类（例如 `BooleanTopic`: [Java](#)），但在 `Topic` 层级不会检查实际主题类型是否匹配。获取特定类型主题的首选方式是调用相应的特定类型获取方法；也可以直接把通用 `Topic` 转换为特定类型的 `Topic` 类。

```
NetworkTableInstance inst = NetworkTableInstance.getDefault();
NetworkTable table = inst.getTable("datatable");
// get a topic from a NetworkTableInstance
// the topic name in this case is the full name
DoubleTopic dblTopic = inst.getDoubleTopic("/datatable/X");
// get a topic from a NetworkTable
// the topic name in this case is the name within the table;
// this line and the one above reference the same topic
DoubleTopic dblTopic = table.getDoubleTopic("X");
// get a type-specific topic from a generic Topic
Topic genericTopic = inst.getTopic("/datatable/X");
DoubleTopic dblTopic = new DoubleTopic(genericTopic);
```

Publishing and Subscribing to a Topic / 发布和订阅主题

Publishing to a Topic / 向主题发布值

In order to create a topic and publish values to it, it's necessary to create a publisher.

要创建主题并向其中发布值，必须创建发布者。

NetworkTable publishers are represented as type-specific Publisher objects (e.g. **BooleanPublisher**: [Java](#)). Publishers are only active as long as the Publisher object exists. Typically you want to keep publishing longer than the local scope of a function, so it's necessary to store the Publisher object somewhere longer term, e.g. in an instance variable. In Java, the `close()` method needs to be called to stop publishing.

NetworkTable 发布者由特定类型的 Publisher 对象表示（例如 **BooleanPublisher**: [Java](#)）。发布者仅在 Publisher 对象存在期间处于活动状态。通常需要发布的时间会超出函数的局部作用域，因此必须把 Publisher 对象保存在生命周期更长的位置，例如实例变量中。在 Java 中，需要调用 `close()` 才能停止发布。

Publishing values is done via a `set()` operation. By default, this operation uses the current time, but a timestamp may optionally be specified. Specifying a timestamp can be useful when multiple values should have the same update timestamp. The timestamp units are integer microseconds (see example code for how to get a current timestamp that is consistent with the library).

通过 `set()` 操作发布值。默认情况下，此操作使用当前时间，但也可以选择指定时间戳。当多个值应使用同一个更新时间戳时，指定时间戳很有用。时间戳的单位是整数微秒（如何取得与该库一致的当前时间戳，请参阅示例代码）。

```
public class Example {
    // the publisher is an instance variable so its lifetime matches that of the class
    final DoublePublisher dblPub;
    public Example(DoubleTopic dblTopic) {
        // start publishing; the return value must be retained (in this case, via
        // an instance variable)
        dblPub = dblTopic.publish();
        // publish options may be specified using PubSubOption
        dblPub = dblTopic.publish(PubSubOption.keepDuplicates(true));
        // publishEx provides additional options such as setting initial
        // properties and using a custom type string. Using a custom type string for
        // types other than raw and string is not recommended. The properties string
        // must be a JSON map.
        dblPub = dblTopic.publishEx("double", "{\"myprop\": 5}");
    }
    public void periodic() {
        // publish a default value
        dblPub.setDefault(0.0);
        // publish a value with current timestamp
        dblPub.set(1.0);
        dblPub.set(2.0, 0); // 0 = use current time
        // publish a value with a specific timestamp; NetworkTablesJNI.now() can
        // be used to get the current time. On the roboRIO, this is the same as
        // the FPGA timestamp (e.g. RobotController.getFPGATime())
        long time = NetworkTablesJNI.now();
        dblPub.set(3.0, time);
        // publishers also implement the appropriate Consumer functional interface;
        // this example assumes void myFunc(DoubleConsumer func) exists
        myFunc(dblPub);
    }
    // often not required in robot code, unless this class doesn't exist for
    // the lifetime of the entire robot program, in which case close() needs to be
    // called to stop publishing
}
```

```

    public void close() {
        // stop publishing
        dblPub.close();
    }
}

```

Subscribing to a Topic / 订阅主题

A subscriber receives value updates made to a topic. Similar to publishers, NetworkTable subscribers are represented as type-specific Subscriber classes (e.g. `BooleanSubscriber`: [Java](#)) that must be stored somewhere to continue subscribing.

订阅者接收对主题所做的值更新。与发布者类似，NetworkTable 订阅者由特定类型的 Subscriber 类表示（例如 `BooleanSubscriber`: [Java](#)）；必须把对象保存在某处，才能持续订阅。

Subscribers have a range of different ways to read received values. It's possible to just read the most recent value using `get()`, read the most recent value, along with its timestamp, using `getAtomic()`, or get an array of all value changes since the last call using `readQueue()` or `readQueueValues()`.

订阅者有多种读取已接收值的方法。可以用 `get()` 只读取最新值；用 `getAtomic()` 同时读取最新值及其时间戳；或者用 `readQueue()` 或 `readQueueValues()` 获取自上次调用以来所有值变化组成的数组。

```

public class Example {
    // the subscriber is an instance variable so its lifetime matches that of the class
    final DoubleSubscriber dblSub;
    public Example(DoubleTopic dblTopic) {
        // start subscribing; the return value must be retained.
        // the parameter is the default value if no value is available when get() is called
        dblSub = dblTopic.subscribe(0.0);
        // subscribe options may be specified using PubSubOption
        dblSub =
            dblTopic.subscribe(0.0, PubSubOption.keepDuplicates(true), PubSubOption.pollStorage(10));
        // subscribeEx provides the options of using a custom type string.
        // Using a custom type string for types other than raw and string is not recommended.
        dblSub = dblTopic.subscribeEx("double", 0.0);
    }
    public void periodic() {
        // simple get of most recent value; if no value has been published,
        // returns the default value passed to the subscribe() function
        double val = dblSub.get();
        // get the most recent value; if no value has been published, returns
        // the passed-in default value
        double val = dblSub.get(-1.0);
        // subscribers also implement the appropriate Supplier interface, e.g. DoubleSupplier
        double val = dblSub.getAsDouble();
        // get the most recent value, along with its timestamp
        TimestampedDouble tsVal = dblSub.getAtomic();
        // read all value changes since the last call to readQueue/readQueueValues
        // readQueue() returns timestamps; readQueueValues() does not.
        TimestampedDouble[] tsUpdates = dblSub.readQueue();
        double[] valUpdates = dblSub.readQueueValues();
    }
    // often not required in robot code, unless this class doesn't exist for
    // the lifetime of the entire robot program, in which case close() needs to be
    // called to stop subscribing
    public void close() {
        // stop subscribing
        dblSub.close();
    }
}

```

Using Entry to Both Subscribe and Publish / 使用 Entry 同时订阅和发布

An entry is a combined publisher and subscriber. The subscriber is always active, but the publisher is not created until a publish operation is performed (e.g. a value is "set", aka published, on the entry). This may be more convenient than maintaining a separate publisher and subscriber. Similar to publishers and subscribers, NetworkTable entries are represented as type-specific Entry classes (e.g. [BooleanEntry: Java](#)) that must be retained to continue subscribing (and publishing).

条目 (entry) 结合了发布者和订阅者。订阅者始终处于活动状态，但只有执行发布操作时（例如在条目上“设置”一个值，即发布该值），才会创建发布者。相比分别维护发布者和订阅者，这可能更方便。与发布者和订阅者类似，NetworkTable 条目由特定类型的 Entry 类表示（例如 [BooleanEntry: Java](#)）；必须保留该对象才能持续订阅（及发布）。

```
public class Example {
    // the entry is an instance variable so its lifetime matches that of the class
    final DoubleEntry dblEntry;
    public Example(DoubleTopic dblTopic) {
        // start subscribing; the return value must be retained.
        // the parameter is the default value if no value is available when get() is called
        dblEntry = dblTopic.getEntry(0.0);
        // publish and subscribe options may be specified using PubSubOption
        dblEntry =
            dblTopic.getEntry(0.0, PubSubOption.keepDuplicates(true), PubSubOption.pollStorage(10));
        // getEntryEx provides the options of using a custom type string.
        // Using a custom type string for types other than raw and string is not recommended.
        dblEntry = dblTopic.getEntryEx("double", 0.0);
    }
    public void periodic() {
        // entries support all the same methods as subscribers:
        double val = dblEntry.get();
        double val = dblEntry.get(-1.0);
        double val = dblEntry.getAsDouble();
        TimestampedDouble tsVal = dblEntry.getAtomic();
        TimestampedDouble[] tsUpdates = dblEntry.readQueue();
        double[] valUpdates = dblEntry.readQueueValues();
        // entries also support all the same methods as publishers; the first time
        // one of these is called, an internal publisher is automatically created
        dblEntry.setDefault(0.0);
        dblEntry.set(1.0);
        dblEntry.set(2.0, 0); // 0 = use current time
        long time = NetworkTablesJNI.now();
        dblEntry.set(3.0, time);
        myFunc(dblEntry);
    }
    public void unpublish() {
        // you can stop publishing while keeping the subscriber alive
        dblEntry.unpublish();
    }
    // often not required in robot code, unless this class doesn't exist for
    // the lifetime of the entire robot program, in which case close() needs to be
    // called to stop subscribing
    public void close() {
        // stop subscribing/publishing
        dblEntry.close();
    }
}
```

Using GenericEntry, GenericPublisher, and GenericSubscriber / 使用 GenericEntry、GenericPublisher 和 GenericSubscriber

For the most robust code, using the type-specific Publisher, Subscriber, and Entry classes is recommended, but in some cases it may be easier to write code that uses type-specific get and set function calls instead of having the NetworkTables

type be exposed via the class (object) type. The `GenericPublisher` ([Java](#)), `GenericSubscriber` ([Java](#)), and `GenericEntry` ([Java](#)) classes enable this approach.

为获得最稳健的代码，建议使用特定类型的 `Publisher`、`Subscriber` 和 `Entry` 类。不过，在某些情况下，使用特定类型的 `get` 和 `set` 方法调用会更方便，而无需通过类（对象）类型体现 `NetworkTables` 类型。`GenericPublisher` ([Java](#))、`GenericSubscriber` ([Java](#)) 和 `GenericEntry` ([Java](#)) 类支持这种方式。

```
public class Example {
    // the entry is an instance variable so its lifetime matches that of the class
    final GenericPublisher pub;
    final GenericSubscriber sub;
    final GenericEntry entry;
    public Example(Topic topic) {
        // start subscribing; the return value must be retained.
        // when publishing, a type string must be provided
        pub = topic.genericPublish("double");
        // subscribing can optionally include a type string
        // unlike type-specific subscribers, no default value is provided
        sub = topic.genericSubscribe();
        sub = topic.genericSubscribe("double");
        // when getting an entry, the type string is also optional; if not provided
        // the publisher data type will be determined by the first publisher-creating call
        entry = topic.getGenericEntry();
        entry = topic.getGenericEntry("double");
        // publish and subscribe options may be specified using PubSubOption
        pub = topic.genericPublish("double",
            PubSubOption.keepDuplicates(true), PubSubOption.pollStorage(10));
        sub =
            topic.genericSubscribe(PubSubOption.keepDuplicates(true), PubSubOption.pollStorage(10));
        entry =
            topic.getGenericEntry(PubSubOption.keepDuplicates(true), PubSubOption.pollStorage(10));
        // genericPublishEx provides the option of setting initial properties.
        pub = topic.genericPublishEx("double", "{\"retained\": true}",
            PubSubOption.keepDuplicates(true), PubSubOption.pollStorage(10));
    }
    public void periodic() {
        // generic subscribers and entries have typed get operations; a default must be provided
        double val = sub.getDouble(-1.0);
        double val = entry.getDouble(-1.0);
        // they also support an untyped get (also meets Supplier<NetworkTableValue> interface)
        NetworkTableValue val = sub.get();
        NetworkTableValue val = entry.get();
        // they also support readQueue
        NetworkTableValue[] updates = sub.readQueue();
        NetworkTableValue[] updates = entry.readQueue();
        // publishers and entries have typed set operations; these return false if the
        // topic already exists with a mismatched type
        boolean success = pub.setDefaultDouble(1.0);
        boolean success = pub.setBoolean(true);
        // they also implement a generic set and Consumer<NetworkTableValue> interface
        boolean success = entry.set(NetworkTableValue.makeDouble(...));
        boolean success = entry.accept(NetworkTableValue.makeDouble(...));
    }
    public void unpublish() {
        // you can stop publishing an entry while keeping the subscriber alive
        entry.unpublish();
    }
    // often not required in robot code, unless this class doesn't exist for
    // the lifetime of the entire robot program, in which case close() needs to be
    // called to stop subscribing/publishing
    public void close() {
        pub.close();
        sub.close();
        entry.close();
    }
}
```

Subscribing to Multiple Topics / 订阅多个主题

While in most cases it's only necessary to subscribe to individual topics, it is sometimes useful (e.g. in dashboard applications) to subscribe and get value updates for changes to multiple topics. Listeners (see [listening for changes](#)) can be used directly, but creating a **MultiSubscriber** ([Java](#)) allows specifying subscription options and reusing the same subscriber for multiple listeners.

虽然多数情况下只需订阅单个主题，但有时（例如仪表盘应用程序）需要订阅多个主题，并获取它们变化时的值更新。可以直接使用监听器（参见[监听变化](#)），但创建 **MultiSubscriber** ([Java](#)) 还可以指定订阅选项，并让多个监听器复用同一个订阅者。

```
public class Example {
    // the subscriber is an instance variable so its lifetime matches that of the class
    final MultiSubscriber multiSub;
    final NetworkTableListenerPoller poller;
    public Example(NetworkTableInstance inst) {
        // start subscribing; the return value must be retained.
        // provide an array of topic name prefixes
        multiSub = new MultiSubscriber(inst, new String[] {"/table1/", "/table2/"});
        // subscribe options may be specified using PubSubOption
        multiSub = new MultiSubscriber(inst, new String[] {"/table1/", "/table2/"},
            PubSubOption.keepDuplicates(true));
        // to get value updates from a MultiSubscriber, it's necessary to create a listener
        // (see the listener documentation for more details)
        poller = new NetworkTableListenerPoller(inst);
        poller.addListener(multiSub, EnumSet.of(NetworkTableEvent.Kind.kValueAll));
    }
    public void periodic() {
        // read value events
        NetworkTableEvent[] events = poller.readQueue();
        for (NetworkTableEvent event : events) {
            NetworkTableValue value = event.valueData.value;
        }
    }
    // often not required in robot code, unless this class doesn't exist for
    // the lifetime of the entire robot program, in which case close() needs to be
    // called to stop subscribing
    public void close() {
        // close listener
        poller.close();
        // stop subscribing
        multiSub.close();
    }
}
```

Publish/Subscribe Options / 发布 / 订阅选项

Publishers and subscribers have various options that affect their behavior. Options can only be set at the creation of the publisher, subscriber, or entry. Options set on an entry affect both the publisher and subscriber portions of the entry. The above examples show how options can be set when creating a publisher or subscriber.

发布者和订阅者具有影响其行为的多种选项。只能在创建发布者、订阅者或条目时设置这些选项。设在条目上的选项会同时影响其发布者和订阅者部分。前面的示例展示了创建发布者或订阅者时如何设置选项。

Subscriber options:

订阅者选项:

- **pollStorage**: Polling storage size for a subscription. Specifies the maximum number of updates NetworkTables should store between calls to the subscriber's `readQueue()` function. If zero, defaults to 1 if `sendAll` is false, 20 if `sendAll` is true.

- `pollStorage`: 订阅的轮询存储容量。指定两次调用订阅者的 `readQueue()` 之间, NetworkTables 最多应存储多少个更新。如果设为零, 当 `sendAll` 为 `false` 时默认使用 1, 为 `true` 时默认使用 20。
- `topicsOnly`: Don't send value changes, only topic announcements. Defaults to false. As a client doesn't get topic announcements for topics it is not subscribed to, this option may be used with `MultiSubscriber` to get topic announcements for a particular topic name prefix, without also getting all value changes.
- `topicsOnly`: 只发送主题公告, 不发送值变化。默认为 `false`。由于客户端不会收到未订阅主题的公告, 可将此选项与 `MultiSubscriber` 一起使用, 以获取特定主题名称前缀的公告, 同时不接收全部值变化。
- `excludePublisher`: Used to exclude a single publisher's updates from being queued to the subscriber's `readQueue()` function. This is primarily useful in scenarios where you don't want local value updates to be "echoed back" to a local subscriber. Regardless of this setting, the topic value is updated--this only affects `readQueue()` on this subscriber.
- `excludePublisher`: 用于防止单个发布者的更新进入订阅者的 `readQueue()` 队列。主要适用于不希望本地值更新“回显”给本地订阅者的场景。不论如何设置, 主题值仍会更新; 此选项只影响该订阅者的 `readQueue()`。
- `disableRemote`: If true, remote value updates are not queued for `readQueue()`. Defaults to false. Regardless of this setting, the topic value is updated--this only affects `readQueue()` on this subscriber.
- `disableRemote`: 为 `true` 时, 远程值更新不会进入 `readQueue()` 队列。默认为 `false`。不论如何设置, 主题值仍会更新; 此选项只影响该订阅者的 `readQueue()`。
- `disableLocal`: If true, local value updates are not queued for `readQueue()`. Defaults to false. Regardless of this setting, the topic value is updated--this only affects `readQueue()` on this subscriber.
- `disableLocal`: 为 `true` 时, 本地值更新不会进入 `readQueue()` 队列。默认为 `false`。不论如何设置, 主题值仍会更新; 此选项只影响该订阅者的 `readQueue()`。

Subscriber and publisher options:

订阅者和发布者共用的选项:

- `periodic`: How frequently changes will be sent over the network, in seconds. NetworkTables may send more frequently than this (e.g. use a combined minimum period for all values) or apply a restricted range to this value. The default is 0.1 seconds. For publishers, it specifies how frequently local changes should be sent over the network; for subscribers, it is a request to the server to send server changes at the requested rate. Note that regardless of the setting of this option, only value changes are sent, unless the `keepDuplicates` option is set.
- `periodic`: 通过网络发送变化的频率, 单位为秒。NetworkTables 可能以比此设置更高的频率发送 (例如, 为所有值采用合并后的最短周期), 也可能将该值限制在允许范围内。默认值为 0.1 秒。对于发布者, 它指定本地变化通过网络发送的频率; 对于订阅者, 它表示向服务器请求按指定频率发送服务器端变化。请注意, 无论此选项如何设置, 除非启用了 `keepDuplicates`, 否则只发送发生变化的值。
- `sendAll`: If true, send all value changes over the network. Defaults to false. As with `periodic`, this is a request to the server for subscribers and a behavior change for publishers.
- `sendAll`: 为 `true` 时, 通过网络发送所有值变化。默认为 `false`。与 `periodic` 一样, 对订阅者而言这是向服务器提出的请求, 对发布者而言则会改变其行为。
- `keepDuplicates`: If true, preserves duplicate value changes (rather than ignoring them). Defaults to false. As with `periodic`, this is a request to the server for subscribers and a behavior change for publishers.
- `keepDuplicates`: 为 `true` 时, 保留重复的值变化, 而不是忽略它们。默认为 `false`。与 `periodic` 一样, 对订阅者而言这是向服务器提出的请求, 对发布者而言则会改变其行为。

Entry options:

条目选项:

- `excludeSelf`: Provides the same behavior as `excludePublisher` for the entry's internal publisher. Defaults to false.
- `excludeSelf`: 对于条目的内部发布者, 提供与 `excludePublisher` 相同的行为。默认为 `false`。

NetworkTableEntry / NetworkTableEntry

`NetworkTableEntry` ([Java](#)) is a class that exists for backwards compatibility. New code should prefer using type-specific `Publisher` and `Subscriber` classes, or `GenericEntry` if non-type-specific access is needed.

`NetworkTableEntry` ([Java](#)) 是为保持向后兼容而存在的类。新代码应优先使用特定类型的 `Publisher` 和 `Subscriber` 类；需要不限定类型的访问时，则使用 `GenericEntry`。

It is similar to `GenericEntry` in that it supports both publishing and subscribing in a single object. However, unlike `GenericEntry`, `NetworkTableEntry` is not released (e.g. unsubscribes/unpublishes) if `close()` is called; instead, it operates similar to `Topic`, in that only a single `NetworkTableEntry` exists for each topic and it lasts for the lifetime of the instance.

它与 `GenericEntry` 相似，都支持在单个对象中发布和订阅。不过，与 `GenericEntry` 不同，调用 `close()` 不会释放 `NetworkTableEntry`（即取消订阅或发布）；它的行为更接近 `Topic`：每个主题只有一个 `NetworkTableEntry`，并在实例的整个生命周期内存在。

NetworkTables Instances / NetworkTables 实例

The NetworkTables implementation supports simultaneous operation of multiple "instances." Each instance has a completely independent set of topics, publishers, subscribers, and client/server state. This feature is mainly useful for unit testing. It allows a single program to be a member of two NetworkTables "networks" that contain different (and unrelated) sets of topics, or running both client and server instances in a single program.

NetworkTables 实现支持多个“实例”同时运行。每个实例都拥有完全独立的一组主题、发布者、订阅者以及客户端 / 服务器状态。此功能主要用于单元测试。它允许单个程序加入两个包含不同且互不相关主题集合的 NetworkTables“网络”，也可以在单个程序内同时运行客户端实例和服务器实例。

For most general usage, you should use the "default" instance, as all current dashboard programs can only connect to a single NetworkTables server at a time. Normally the default instance is set up on the robot as a server, and used for communication with the dashboard program running on your driver station computer. This is what the SmartDashboard and LiveWindow classes use.

在大多数一般用途下，应使用“默认”实例，因为目前所有仪表板程序一次只能连接一台 NetworkTables 服务器。通常，默认实例在机器人上作为服务器运行，用于与 Driver Station 计算机上运行的仪表板程序通信。SmartDashboard 和 LiveWindow 类使用的就是这个实例。

However, if you wanted to do unit testing of your robot program's NetworkTables communications, you could set up your unit tests such that they create a separate client instance (still within the same program) and have it connect to the server instance that the main robot code is running.

不过，如果要对机器人程序的 NetworkTables 通信进行单元测试，可以让单元测试创建一个独立的客户端实例（仍在同一程序中），并将其连接到主机器人代码运行的服务器实例。

The `NetworkTableInstance` ([Java](#)) class provides the API abstraction for instances. The number of instances that can be simultaneously created is limited to 16 (including the default instance), so when using multiple instances in cases such as unit testing code, it's important to destroy instances that are no longer needed.

`NetworkTableInstance` ([Java](#)) 类提供实例的 API 抽象。可同时创建的实例最多为 16 个（包括默认实例），因此在单元测试代码等使用多个实例的场景中，应销毁不再需要的实例。

Destroying a `NetworkTableInstance` frees all resources related to the instance. All classes that reference the instance (e.g. Topics, Publishers, and Subscribers) are invalidated and may result in unexpected behavior if used after the instance is destroyed.

销毁 `NetworkTableInstance` 会释放与该实例有关的所有资源。引用该实例的所有类（例如 Topics、Publishers 和 Subscribers）都会失效；如果在实例销毁后继续使用它们，可能出现意外行为。

```
// get the default NetworkTable instance
NetworkTableInstance defaultInst = NetworkTableInstance.getDefault();
// create a NetworkTable instance
NetworkTableInstance inst = NetworkTableInstance.create();
// destroy a NetworkTable instance
inst.close();
```

NetworkTables Networking / NetworkTables 网络通信

The advantage of the robot program being the server is that it's at a known network name (and typically at a known address) that is based on the team number. This is why it's possible in both the NetworkTables client API and in most dashboards to simply provide the team number, rather than a server address. As the robot program is the server, note this means the NetworkTables server is running on the local computer when running in simulation.

让机器人程序担任服务器的好处是：它具有基于队伍号码确定的已知网络名称（通常也有已知地址）。因此，在 NetworkTables 客户端 API 和大多数仪表板中，只需提供队伍号码，而无需提供服务器地址。由于机器人程序是服务器，请注意，在仿真运行时，NetworkTables 服务器运行于本地计算机。

Starting a NetworkTables Server / 启动 NetworkTables 服务器

```
NetworkTableInstance inst = NetworkTableInstance.getDefault();
inst.startServer();
```

Starting a NetworkTables Client / 启动 NetworkTables 客户端

```
NetworkTableInstance inst = NetworkTableInstance.getDefault();
// start a NT4 client
inst.startClient4("example client");
// connect to a roboRIO with team number TEAM
inst.setServerTeam(TEAM);
// starting a DS client will try to get the roboRIO address from the DS application
inst.startDSClient();
// connect to a specific host/port
inst.setServer("host", NetworkTableInstance.kDefaultPort4)
```

Listening for Changes / 监听变化

A common use case for NetworkTables is where a coprocessor generates values that need to be sent to the robot. For example, imagine that some image processing code running on a coprocessor computes the heading and distance to a goal and sends those values to the robot. In this case it might be desirable for the robot program to be notified when new values arrive.

NetworkTables 的一个常见用途是将协处理器生成的值发送给机器人。例如，假设协处理器上运行的图像处理代码计算目标的朝向和距离，并把这些值发给机器人。此时，机器人程序可能希望在新值到达时收到通知。

There are a few different ways to detect that a topic's value has changed; the easiest way is to periodically call a subscriber's `get()`, `readQueue()`, or `readQueueValues()` function from the robot's periodic loop, as shown below:

检测主题值变化有几种方法；最简单的方法是在机器人的周期循环中定期调用订阅者的 `get()`、`readQueue()` 或 `readQueueValues()`，如下所示：

```
public class Example {
    final DoubleSubscriber ySub;
    double prev;
    public Example() {
        // get the default instance of NetworkTables
        NetworkTableInstance inst = NetworkTableInstance.getDefault();
        // get the subtable called "datatable"
        NetworkTable datatable = inst.getTable("datatable");
        // subscribe to the topic in "datatable" called "Y"
        ySub = datatable.getDoubleTopic("Y").subscribe(0.0);
    }
    public void periodic() {
        // get() can be used with simple change detection to the previous value
        double value = ySub.get();
        if (value != prev) {
            prev = value; // save previous value
            System.out.println("X changed value: " + value);
        }
        // readQueueValues() provides all value changes since the last call;
        // this way it's not possible to miss a change by polling too slowly
        for (double iterVal : ySub.readQueueValues()) {
            System.out.println("X changed value: " + iterVal);
        }
        // readQueue() is similar to readQueueValues(), but provides timestamps
        // for each change as well
        for (TimestampedDouble tsValue : ySub.readQueue()) {
            System.out.println("X changed value: " + tsValue.value + " at local time " + tsValue.timestamp);
        }
    }
    // may not be necessary for robot programs if this class lives for
    // the length of the program
    public void close() {
        ySub.close();
    }
}
```

With a command-based robot, it's also possible to use `NetworkBooleanEvent` to link boolean topic changes to callback actions (e.g. running commands).

对于 Command-Based 机器人，还可以使用 `NetworkBooleanEvent` 将布尔主题的变化与回调动作（例如运行 Command）关联起来。

While these functions suffice for value changes on a single topic, they do not provide insight into changes to topics (when a topic is published or unpublished, or when a topic's properties change) or network connection changes (e.g. when a

client connects or disconnects). They also don't provide a way to get in-order updates for value changes across multiple topics. For these needs, NetworkTables provides an event listener facility.

这些方法足以处理单个主题的值变化，但无法获知主题本身的变化（主题发布、取消发布或属性变更）以及网络连接变化（例如客户端连接或断开）。它们也无法按顺序获取跨多个主题的值更新。针对这些需求，NetworkTables 提供了事件监听器机制。

The easiest way to use listeners is via `NetworkTableInstance`. For more automatic control over listener lifetime, and to operate without a background thread, NetworkTables also provides separate classes for both polled listeners (`NetworkTableListenerPoller`), which store events into an internal queue that must be periodically read to get the queued events, and threaded listeners (`NetworkTableListener`), which call a callback function from a background thread.

使用监听器最简单的方式是通过 `NetworkTableInstance`。为了更自动地管理监听器生命周期，以及在不使用后台线程的情况下运行，NetworkTables 还分别提供了轮询式监听器类 (`NetworkTableListenerPoller`) 和线程式监听器类 (`NetworkTableListener`)：前者把事件存入必须定期读取的内部队列，后者从后台线程调用回调函数。

NetworkTableEvent / NetworkTableEvent

All listener callbacks take a single `NetworkTableEvent` parameter, and similarly, reading a listener poller returns an array of `NetworkTableEvent`. The event contains information including what kind of event it is (e.g. a value update, a new topic, a network disconnect), the handle of the listener that caused the event to be generated, and more detailed information that depends on the type of the event (connection information for connection events, topic information for topic-related events, value data for value updates, and the log message for log message events).

所有监听器回调都接收一个 `NetworkTableEvent` 参数；类似地，读取监听器轮询器会返回一个 `NetworkTableEvent` 数组。事件包含事件类型（例如值更新、新主题或网络断开）、触发该事件的监听器句柄，以及取决于事件类型的更详细信息：连接事件的连接信息、主题相关事件的主题信息、值更新的值数据，或日志消息事件的日志消息。

Using NetworkTableInstance to Listen for Changes / 使用 NetworkTableInstance 监听变化

The below example listens to various kinds of events using `NetworkTableInstance`. The listener callback provided to any of the `addListener` functions will be called asynchronously from a background thread when a matching event occurs.

以下示例通过 `NetworkTableInstance` 监听多种事件。传给任何 `addListener` 方法的监听器回调，都会在匹配事件发生时从后台线程异步调用。

WARNING: Because the listener callback is called from a separate background thread, it's important to use thread-safe synchronization approaches such as mutexes or atomics to pass data to/from the main code and the listener callback function.

警告： 由于监听器回调是在独立的后台线程中调用的，必须采用原子变量等线程安全的同步方式，在主代码与监听器回调函数之间传递数据。

The `addListener` functions in `NetworkTableInstance` return a listener handle. This can be used to remove the listener later.

`NetworkTableInstance` 中的 `addListener` 方法会返回监听器句柄，之后可用它移除监听器。

```
public class Example {
    final DoubleSubscriber ySub;
    // use an AtomicReference to make updating the value thread-safe
```

```

final AtomicReference<Double> yValue = new AtomicReference<Double>();
// retain listener handles for later removal
int connListenerHandle;
int valueListenerHandle;
int topicListenerHandle;
public Example() {
    // get the default instance of NetworkTables
    NetworkTableInstance inst = NetworkTableInstance.getDefault();
    // add a connection listener; the first parameter will cause the
    // callback to be called immediately for any current connections
    connListenerHandle = inst.addConnectionListener(true, event -> {
        if (event.is(NetworkTableEvent.Kind.kConnected)) {
            System.out.println("Connected to " + event.connInfo.remote_id);
        } else if (event.is(NetworkTableEvent.Kind.kDisconnected)) {
            System.out.println("Disconnected from " + event.connInfo.remote_id);
        }
    });
    // get the subtable called "datatable"
    NetworkTable datatable = inst.getTable("datatable");
    // subscribe to the topic in "datatable" called "Y"
    ySub = datatable.getDoubleTopic("Y").subscribe(0.0);
    // add a listener to only value changes on the Y subscriber
    valueListenerHandle = inst.addListener(
        ySub,
        EnumSet.of(NetworkTableEvent.Kind.kValueAll),
        event -> {
            // can only get doubles because it's a DoubleSubscriber, but
            // could check value.isDouble() here too
            yValue.set(event.valueData.value.getDouble());
        });
    // add a listener to see when new topics are published within datatable
    // the string array is an array of topic name prefixes.
    topicListenerHandle = inst.addListener(
        new String[] { datatable.getPath() + "/" },
        EnumSet.of(NetworkTableEvent.Kind.kTopic),
        event -> {
            if (event.is(NetworkTableEvent.Kind.kPublish)) {
                // topicInfo.name is the full topic name, e.g. "/datatable/X"
                System.out.println("newly published " + event.topicInfo.name);
            }
        });
}
public void periodic() {
    // get the latest value by reading the AtomicReference; set it to null
    // when we read to ensure we only get value changes
    Double value = yValue.getAndSet(null);
    if (value != null) {
        System.out.println("got new value " + value);
    }
}
// may not be needed for robot programs if this class exists for the
// lifetime of the program
public void close() {
    NetworkTableInstance inst = NetworkTableInstance.getDefault();
    inst.removeListener(topicListenerHandle);
    inst.removeListener(valueListenerHandle);
    inst.removeListener(connListenerHandle);
    ySub.close();
}
}

```

Writing a Simple NetworkTables Robot Program / 编写简单的 NetworkTables 机器人程序

In a robot program, a NetworkTables server is automatically started on the default instance. So it's only necessary to get the default instance to start publishing or subscribing and have it visible over the network.

在机器人程序中，默认实例上会自动启动 NetworkTables 服务器。因此，只需获取默认实例，就可以开始发布或订阅，并使相应内容在网络上可见。

The example robot program below publishes incrementing X and Y values to a table named `datatable`. The values for X and Y can be easily viewed using the OutlineViewer program that shows the NetworkTables hierarchy and all the values associated with each topic.

下面的机器人程序示例向名为 `datatable` 的表发布不断增大的 X 和 Y 值。使用 OutlineViewer 程序可以方便地查看 X 和 Y 的值；该程序会显示 NetworkTables 的层级结构以及与每个主题关联的全部值。

```
package edu.wpi.first.wpilibj.templates;
import edu.wpi.first.wpilibj.TimedRobot;
import edu.wpi.first.networktables.DoublePublisher;
import edu.wpi.first.networktables.NetworkTable;
import edu.wpi.first.networktables.NetworkTableInstance;
public class EasyNetworkTableExample extends TimedRobot {
    DoublePublisher xPub;
    DoublePublisher yPub;

    public Robot() {
        // Get the default instance of NetworkTables that was created automatically
        // when the robot program starts
        NetworkTableInstance inst = NetworkTableInstance.getDefault();
        // Get the table within that instance that contains the data. There can
        // be as many tables as you like and exist to make it easier to organize
        // your data. In this case, it's a table called datatable.
        NetworkTable table = inst.getTable("datatable");
        // Start publishing topics within that table that correspond to the X and Y values
        // for some operation in your program.
        // The topic names are actually "/datatable/x" and "/datatable/y".
        xPub = table.getDoubleTopic("x").publish();
        yPub = table.getDoubleTopic("y").publish();
    }

    double x = 0;
    double y = 0;
    public void teleopPeriodic() {
        // Publish values that are constantly increasing.
        xPub.set(x);
        yPub.set(y);
        x += 0.05;
        y += 1.0;
    }
}
```

译注：原 Java 示例中的类名为 `EasyNetworkTableExample`，构造函数却写为 `Robot()`；代码按本地文档原样保留。

Creating a Client-side Program / 创建客户端程序

If all you need to do is have your robot program communicate with a COTS coprocessor or a dashboard running on the Driver Station laptop, then the previous examples of writing robot programs are sufficient. But if you would like to write some custom client code that would run on the drivers station or on a coprocessor then you need to know how to build NetworkTables programs for those (non-roboRIO) platforms.

如果只需让机器人程序与商用现成（COTS）协处理器或运行在 Driver Station 笔记本电脑上的仪表板通信，前面的机器人程序示例已经足够。但如果希望编写在 Driver Station 或协处理器上运行的自定义客户端代码，就需要了解如何为这些非 roboRIO 平台构建 NetworkTables 程序。

A basic client program looks like the following example.

一个基本客户端程序如下面的示例所示。

```
import edu.wpi.first.networktables.DoubleSubscriber;
import edu.wpi.first.networktables.NetworkTable;
import edu.wpi.first.networktables.NetworkTableInstance;
import edu.wpi.first.networktables.NetworkTablesJNI;
import edu.wpi.first.util.CombinedRuntimeLoader;
import java.io.IOException;
import edu.wpi.first.cscore.CameraServerJNI;
import edu.wpi.first.math.WPIMathJNI;
import edu.wpi.first.util.WPIUtilJNI;
public class Program {
    public static void main(String[] args) throws IOException {
        NetworkTablesJNI.Helper.setExtractOnStaticLoad(false);
        WPIUtilJNI.Helper.setExtractOnStaticLoad(false);
        WPIMathJNI.Helper.setExtractOnStaticLoad(false);
        CameraServerJNI.Helper.setExtractOnStaticLoad(false);
        CombinedRuntimeLoader.loadLibraries(Program.class, "wpiutiljni", "wpimathjni", "ntcorejni",
            Core.NATIVE_LIBRARY_NAME, "cscorejni");
        new Program().run();
    }
    public void run() {
        NetworkTableInstance inst = NetworkTableInstance.getDefault();
        NetworkTable table = inst.getTable("datatable");
        DoubleSubscriber xSub = table.getDoubleTopic("x").subscribe(0.0);
        DoubleSubscriber ySub = table.getDoubleTopic("y").subscribe(0.0);
        inst.startClient4("example client");
        inst.setServerTeam(TEAM); // where TEAM=190, 294, etc, or use inst.setServer("hostname") or similar
        inst.startDSClient(); // recommended if running on DS computer; this gets the robot IP from the DS
        while (true) {
            try {
                Thread.sleep(1000);
            } catch (InterruptedException ex) {
                System.out.println("interrupted");
                return;
            }
            double x = xSub.get();
            double y = ySub.get();
            System.out.println("X: " + x + " Y: " + y);
        }
    }
}
```

In this example an instance of NetworkTables is created and subscribers are created to reference the values of "x" and "y" from a table called "datatable".

此示例创建了一个 NetworkTables 实例，并创建订阅者以引用名为 `datatable` 的表中的 `x` 和 `y` 值。

Then this instance is started as a NetworkTables client with the team number (the roboRIO is always the server). Additionally, if the program is running on the Driver Station computer, by using the `startDSClient()` method, NetworkTables will get the robot IP address from the Driver Station.

随后，以队伍号码启动该实例，使其作为 NetworkTables 客户端运行（roboRIO 始终是服务器）。另外，如果程序在 Driver Station 计算机上运行，调用 `startDSClient()` 可让 NetworkTables 从 Driver Station 获取机器人 IP 地址。

Then this sample program simply loops once a second and gets the values for x and y and prints them on the console. In a more realistic program, the client might be processing or generating values for the robot to consume.

接着，此示例程序每秒循环一次，读取 `x` 和 `y` 的值并将其打印到控制台。在更实际的程序中，客户端可能会处理或生成供机器人使用的值。

Building using Gradle / 使用 Gradle 构建

Example build.gradle files are provided in the [StandaloneAppSamples Repository](#). Update the GradleRIO version to correspond to the desired WPILib version.

[StandaloneAppSamples 仓库](#)提供了 `build.gradle` 示例文件。请将 GradleRIO 版本更新为与目标 WPILib 版本对应的版本。

译注：本地嵌入的 `build.gradle` 示例仍写有 GradleRIO 2025.1.1；用于其他赛季时应按原文要求更新版本。代码块保持原样。

```
plugins {
    id "java"
    id 'application'
    id 'com.gradleup.shadow' version '8.3.5'
    id "edu.wpi.first.GradleRIO" version "2025.1.1"
    id 'edu.wpi.first.WpilibTools' version '2.1.0'
}

application {
    mainClass = 'Program'
}

wpilibTools.deps.wpilibVersion = wpi.versions.wpilibVersion.get()

def nativeConfigName = 'wpilibNatives'
def nativeConfig = configurations.create(nativeConfigName)

def nativeTasks = wpilibTools.createExtractionTasks {
    configurationName = nativeConfigName
}

nativeTasks.addToSourceSetResources(sourceSets.main)
nativeConfig.dependencies.add wpilibTools.deps.wpilib("wpimath")
nativeConfig.dependencies.add wpilibTools.deps.wpilib("wpinet")
nativeConfig.dependencies.add wpilibTools.deps.wpilib("wpiutil")
nativeConfig.dependencies.add wpilibTools.deps.wpilib("ntcore")
nativeConfig.dependencies.add wpilibTools.deps.wpilib("cscore")
nativeConfig.dependencies.add wpilibTools.deps.wpilibOpenCv("frc" + wpi.frcYear.get(),
wpi.versions.opencvVersion.get())

dependencies {
    implementation wpilibTools.deps.wpilibJava("wpiutil")
    implementation wpilibTools.deps.wpilibJava("wpimath")
    implementation wpilibTools.deps.wpilibJava("wpinet")
    implementation wpilibTools.deps.wpilibJava("ntcore")
    implementation wpilibTools.deps.wpilibJava("cscore")
    implementation wpilibTools.deps.wpilibJava("cameraserver")
}
```

```

        implementation wpiLibTools.deps.wpiLibOpenCvJava("frc" + wpi.frcYear.get(), wpi.versions.opencvVersion.get())

        implementation group: "com.fasterxml.jackson.core", name: "jackson-annotations", version:
wpi.versions.jacksonVersion.get()
        implementation group: "com.fasterxml.jackson.core", name: "jackson-core", version:
wpi.versions.jacksonVersion.get()
        implementation group: "com.fasterxml.jackson.core", name: "jackson-databind", version:
wpi.versions.jacksonVersion.get()

        implementation group: "org.ejml", name: "ejml-simple", version: wpi.versions.ejmlVersion.get()
        implementation group: "us.hebi.quickbuf", name: "quickbuf-runtime", version:
wpi.versions.quickbufVersion.get();
    }

    shadowJar {
        archiveBaseName = "TestApplication"
        archiveVersion = ""
        exclude("module-info.class")
        archiveClassifier.set(wpiLibTools.currentPlatform.platformName)
    }

    wrapper {
        gradleVersion = '8.11'
    }

```

Migrating from NetworkTables 3.0 to NetworkTables 4.0 / 从 NetworkTables 3.0 迁移到 NetworkTables 4.0

NetworkTables 4.0 (new for 2023) has a number of significant API breaking changes from NetworkTables 3.0, the version of NetworkTables used from 2016-2022.

NetworkTables 4.0 (2023 年引入) 相较于 2016 至 2022 年使用的 NetworkTables 3.0, 包含多项重要的破坏性 API 变更。

NetworkTableEntry / NetworkTableEntry

While `NetworkTableEntry` can still be used (for backwards compatibility), users are encouraged to migrate to use of type-specific Publisher/Subscriber/Entry classes as appropriate, or if necessary, `GenericEntry` (see [publishing and subscribing to a topic](#)). It's important to note that unlike `NetworkTableEntry`, these classes need to have appropriate lifetime management. Some functionality (e.g. persistent settings) has also moved to `Topic` properties (see [networktables tables and topics](#)).

虽然仍可为保持向后兼容而使用 `NetworkTableEntry`, 但建议根据需要迁移到特定类型的 Publisher、Subscriber、Entry 类, 或在必要时使用 `GenericEntry` (参见[发布和订阅主题](#))。需要注意的是, 与 `NetworkTableEntry` 不同, 这些类必须正确管理生命周期。部分功能 (例如持久化设置) 也已移至 `Topic` 属性 (参见[NetworkTables 的表与主题](#))。

NT3 code (was):

原 NT3 代码:

```
public class Example {
    final NetworkTableEntry yEntry;
    final NetworkTableEntry outEntry;
    public Example() {
        NetworkTableInstance inst = NetworkTableInstance.getDefault();
        // get the subtable called "datatable"
        NetworkTable datatable = inst.getTable("datatable");
        // get the entry in "datatable" called "Y"
        yEntry = datatable.getEntry("Y");
        // get the entry in "datatable" called "Out"
        outEntry = datatable.getEntry("Out");
    }
    public void periodic() {
        // read a double value from Y, and set Out to that value multiplied by 2
        double value = yEntry.getDouble(0.0); // default to 0
        outEntry.setDouble(value * 2);
    }
}
```

Recommended NT4 equivalent (should be):

建议使用的 NT4 等效代码:

```
public class Example {
    final DoubleSubscriber ySub;
    final DoublePublisher outPub;
    public Example() {
        NetworkTableInstance inst = NetworkTableInstance.getDefault();
        // get the subtable called "datatable"
        NetworkTable datatable = inst.getTable("datatable");
        // subscribe to the topic in "datatable" called "Y"
    }
}
```

```

// default value is 0
ySub = datatable.getDoubleTopic("Y").subscribe(0.0);
// publish to the topic in "datatable" called "Out"
outPub = datatable.getDoubleTopic("Out").publish();
}
public void periodic() {
    // read a double value from Y, and set Out to that value multiplied by 2
    double value = ySub.get();
    outPub.set(value * 2);
}
// often not required in robot code, unless this class doesn't exist for
// the lifetime of the entire robot program, in which case close() needs to be
// called to stop subscribing
public void close() {
    ySub.close();
    outPub.close();
}
}
}

```

Shuffleboard / Shuffleboard

In WPILib's Shuffleboard classes, usage of `NetworkTableEntry` has been replaced with use of `GenericEntry`.

在 WPILib 的 Shuffleboard 类中, `NetworkTableEntry` 已改为使用 `GenericEntry`。

Force Set Operations / 强制设置操作

Force set operations have been removed, as it's no longer possible to change a topic's type once it's been published. In most cases calls to `forceSet` can simply be replaced with `set`, but more complex scenarios may require a different design approach (e.g. splitting into different topics).

强制设置操作已移除, 因为主题一旦发布, 就不能再改变其类型。在大多数情况下, 可直接用 `set` 替换 `forceSet` 调用; 但更复杂的场景可能需要采用不同的设计 (例如拆分为不同主题)。

Listeners / 监听器

The separate connection, value, and log listeners/events have been unified into a single listener/event. The NetworkTable-level listeners have also been removed. Listeners in many cases can be replaced with subscriber `readQueue()` calls, but if listeners are still required, they can be used via `NetworkTableInstance` (see [listening for changes](#) for more information).

原先分开的连接、值和日志监听器 / 事件已统一为单一的监听器 / 事件。NetworkTable 层级的监听器也已移除。在许多情况下, 可用订阅者的 `readQueue()` 调用替代监听器; 如果仍需要监听器, 可以通过 `NetworkTableInstance` 使用 (详情参见[监听变化](#))。

While NetworkTable-level listeners were removed, table-specific listening functionality can be achieved by using `NetworkTableInstance.addListener()` with a topic name prefix filter. By passing an array of topic name prefixes (such as `["/datatable/"]`), the listener will only receive events for topics within that table. This is demonstrated in the [using networktableinstance to listen for changes](#) section.

虽然 NetworkTable 层级的监听器已移除, 但仍可通过 `NetworkTableInstance.addListener()` 配合主题名称前缀过滤器实现特定表的监听。传入主题名称前缀数组 (例如 `["/datatable/"]`) 后, 监听器只会收到该表中主题的事件。[使用 NetworkTableInstance 监听变化](#) 一节演示了这种用法。

Client/Server Operations / 客户端 / 服务器操作

Starting a NetworkTable server now requires specifying both the NT3 port and the NT4 port. For a NT4-only server, the NT3 port can be specified as 0.

现在启动 NetworkTable 服务器时，必须同时指定 NT3 端口和 NT4 端口。对于仅使用 NT4 的服务器，可将 NT3 端口指定为 0。

A NetworkTable client can only operate in NT3 mode or NT4 mode, not both (there is no provision for automatic fallback). As such, the `startClient()` call has been replaced by `startClient3()` and `startClient4()`. The client must also specify a unique name for itself--the server will reject connection attempts with duplicate names.

NetworkTable 客户端只能在 NT3 或 NT4 模式下运行，不能同时使用两者（没有自动回退机制）。因此，`startClient()` 已分别由 `startClient3()` 和 `startClient4()` 取代。客户端还必须为自己指定唯一名称；服务器会拒绝名称重复的连接请求。

Reading Array Values Published by NetworkTables / 读取通过 NetworkTables 发布的数组值

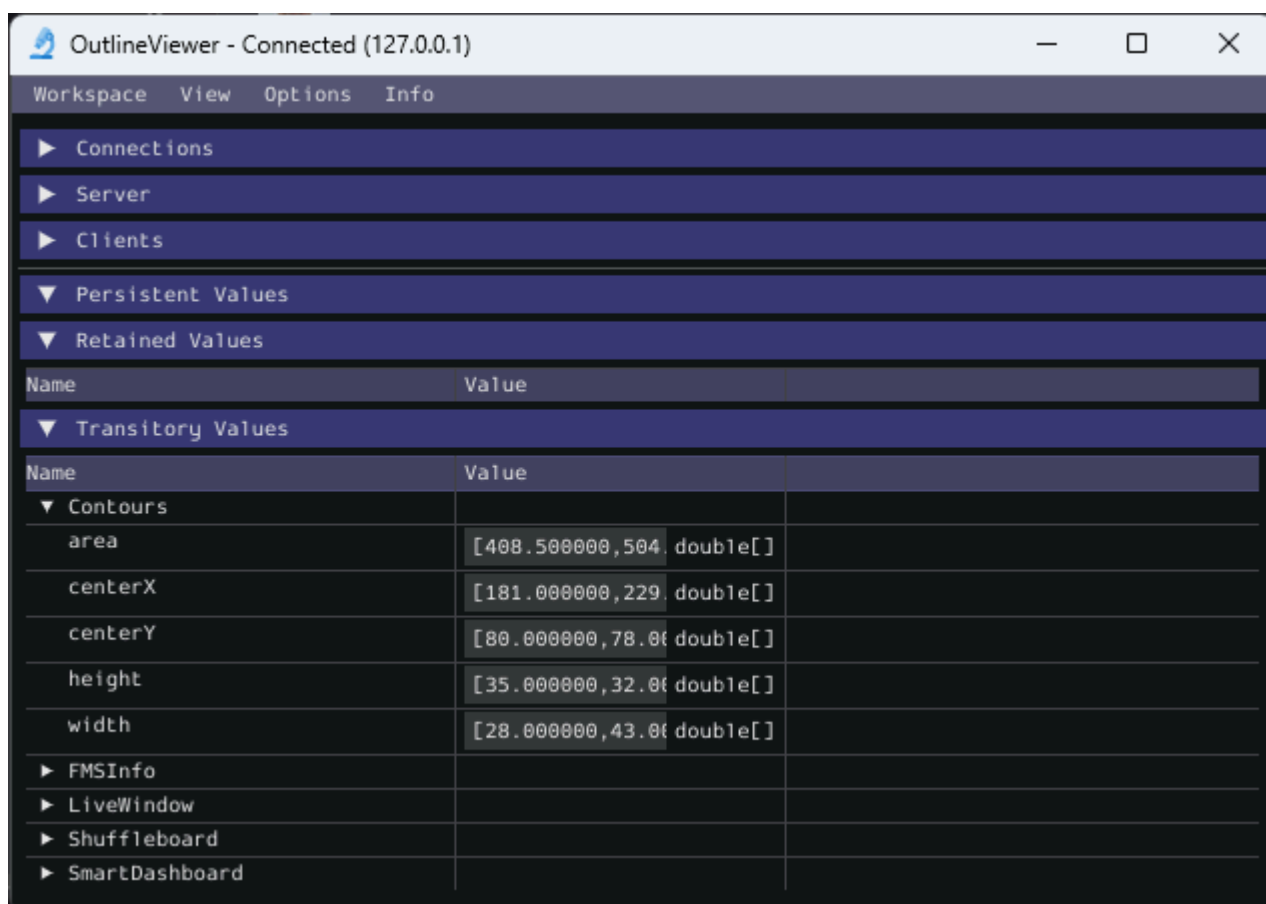
This article describes how to read values published by NetworkTables using a program running on the robot. This is useful when using computer vision where the images are processed on your driver station laptop and the results stored into NetworkTables possibly using a separate vision processor like a raspberry pi, or an image processing tool on the robot.

本文介绍如何在机器人上运行的程序中读取由 NetworkTables 发布的值。当在 Driver Station 笔记本电脑上处理图像，并将结果存入 NetworkTables 时，这种方法很有用；处理工作也可能由 Raspberry Pi 等独立视觉处理器，或机器人上的图像处理程序完成。

Very often the values are for one or more areas of interest such as goals or game pieces and multiple instances are returned. In the example below, several x, y, width, height, and areas are returned by the image processor and the robot program can sort out which of the returned values are interesting through further processing.

这些值通常对应目标或游戏道具等一个或多个感兴趣区域，并会返回多个实例。在下面的示例中，图像处理器返回多个 x、y、宽度、高度和面积值；机器人程序可以进一步处理这些返回值，筛选出感兴趣的结果。

Verify the NetworkTables Topics Being Published / 确认正在发布的 NetworkTables 主题



OutlineViewer - Connected (127.0.0.1)		
Workspace View Options Info		
▶ Connections		
▶ Server		
▶ Clients		
▼ Persistent Values		
▼ Retained Values		
Name	Value	
▼ Transitory Values		
Name	Value	
▼ Contours		
area	[408.500000, 504.000000]	double[]
centerX	[181.000000, 229.000000]	double[]
centerY	[80.000000, 78.000000]	double[]
height	[35.000000, 32.000000]	double[]
width	[28.000000, 43.000000]	double[]
▶ FMSInfo		
▶ LiveWindow		
▶ Shuffleboard		
▶ SmartDashboard		

Image of OutlineViewer with the NetworkTables topics / 显示 NetworkTables 主题的 OutlineViewer 图像

You can verify the names of the NetworkTables topics used for publishing the values by using the Outline Viewer application. It is a C++ program in your user directory in the `wpiLib/<YEAR>/tools` folder. The application is started by selecting the "WPILib" menu in Visual Studio Code then Start Tool then "OutlineViewer".

可以使用 OutlineViewer 应用程序确认用于发布这些值的 NetworkTables 主题名称。它是位于用户目录 `wpiLib/<YEAR>/tools` 文件夹中的 C++ 程序。在 Visual Studio Code 中依次选择“WPILib”菜单、“Start Tool”和“OutlineViewer”即可启动。

In this case the values are stored in a table called GRIP and a sub-table called myContoursReport. You can see that the values are in brackets and there are 2 values in this case for each topic. The NetworkTables topic names are centerX, centerY, area, height, and width.

此处的值保存在名为 `GRIP` 的表及其子表 `myContoursReport` 中。可以看到值位于方括号中，而且本例中每个主题都有 2 个值。NetworkTables 主题名称分别是 `centerX`、`centerY`、`area`、`height` 和 `width`。

The following example is an extremely simplified program that just illustrates the use of NetworkTables. All the code is in the `robotInit()` method so it's only run when the program starts up. In your programs, you would more likely get the values in code that is evaluating which direction to aim the robot in a command or a control loop during the autonomous or teleop periods.

下面的示例程序经过高度简化，仅用于说明 NetworkTables 的用法。所有代码都位于 `robotInit()` 方法中，因此只在程序启动时运行。在实际程序中，更可能在自主阶段或遥控阶段的 Command 或控制循环里读取这些值，以判断机器人应朝哪个方向瞄准。

译注：本地 Java 示例实际在构造函数中创建订阅者，并在 `teleopPeriodic()` 中读取数组；原文关于代码全部位于 `robotInit()` 的描述与示例不一致。

Writing a Program to Access the Topics / 编写访问主题的程序

```
DoubleArraySubscriber areasSub;

public Robot() {
    NetworkTable table = NetworkTableInstance.getDefault().getTable("GRIP/mycontoursReport");
    areasSub = table.getDoubleArrayTopic("area").subscribe(new double[] {});
}

@Override
public void teleopPeriodic() {
    double[] areas = areasSub.get();
    System.out.print("areas: ");
    for (double area : areas) {
        System.out.print(area + " ");
    }
    System.out.println();
}
```

The steps to getting the values and, in this program, printing them are:

获取这些值并在本程序中将它们打印出来的步骤如下：

1. Declare the table variable that will hold the instance of the subtable that have the values.
声明用于保存包含这些值的子表实例的表变量。
2. Initialize the subtable instance so that it can be used later for retrieving the values.
初始化子表实例，以便稍后用它获取这些值。
3. Read the array of values from NetworkTables. In the case of a communicating programs, it's possible that the program producing the output being read here might not yet be available when the robot program starts up. To avoid

issues of the data not being ready, a default array of values is supplied. This default value will be returned if the NetworkTables topic hasn't yet been published. This code will loop over the value of areas every 20ms. 从 NetworkTables 读取值数组。在程序通信过程中，机器人程序启动时，生成此处所读取输出的程序可能尚未就绪。为避免数据尚未准备好而引发问题，需要提供一个默认值数组。如果 NetworkTables 主题尚未发布，就会返回这个默认值。此代码每 20ms 遍历一次 **areas** 的值。

Program Output / 程序输出

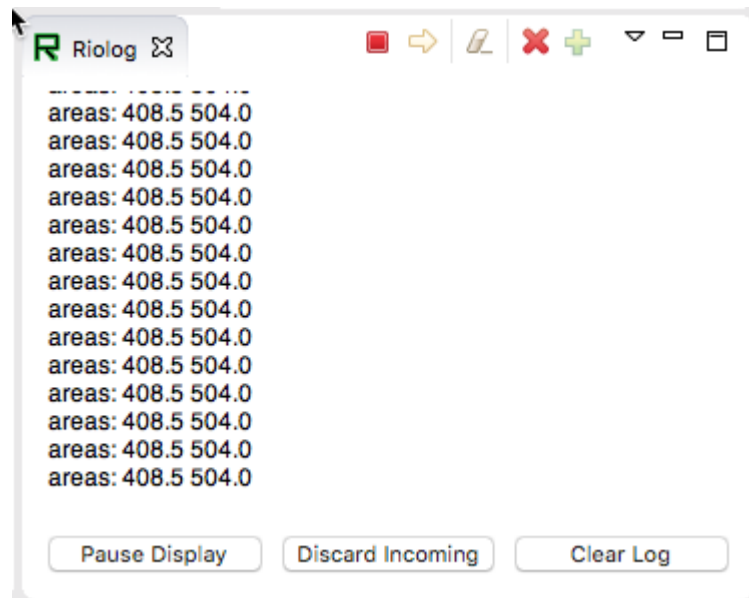


Image of Riolog showing the values / 显示这些值的 Riolog 图像

In this case the program is only looking at the array of areas, but in a real example all the values would more likely be used. Using the Riolog in VS Code or the Driver Station log you can see the values as they are retrieved. This program is using a sample static image so they areas don't change, but you can imagine with a camera on your robot, the values would be changing constantly.

本例只查看面积数组，但在实际应用中，通常还会使用其他所有值。通过 VS Code 中的 Riolog 或 Driver Station 日志，可以看到读取到的值。此程序使用静态示例图像，所以面积值不发生变化；如果机器人上安装摄像头，值就会不断变化。